

TECNOLOGÍAS DE LA INFORMACIÓN
EN LINEA
INGENIERÍA DE SOFTWARE

3 créditos



Profesor Autor:

Ing. Freddy Malla García, Mtr.

Titulaciones	Semestre
INGENIERÍA DE SOFTWARE	SEXTO

Tutorías: El profesor asignado publicará en el entorno virtual de aprendizaje online.utm.edu.ec, y sus horarios de conferencias se indicarán en la sección **CRONOGRAMA**.

PERIODO ABRIL - AGOSTO 2026



Contenido

UNIDAD IV METODOLOGÍAS ÁGILES	1
1. ¿QUÉ ES LA AGILIDAD?	1
1.1. ¿QUÉ ES UN PROCESO ÁGIL?	2
1.2. PRINCIPIOS DE AGILIDAD	3
1.3. FACTORES HUMANOS	4
2. METODOLOGÍAS.....	5
2.1. PROGRAMACIÓN EXTREMA (XP).	7
2.1.1. <i>Pruebas en XP</i>	12
2.1.2. <i>El Proceso Xp</i>	15
2.2. SCRUM	20
2.3. PROCESO UNIFICADO ÁGIL (PUA)	22
2.4. OTROS	23
2.4.1. <i>Desarrollo adaptativo de software (DAS)</i>	24
2.4.2. <i>Método de desarrollo de sistemas dinámicos (MDSD)</i>	26
2.4.3. <i>Cristal</i>	27
2.4.4. <i>Desarrollo impulsado por las características (DIC)</i>	28
2.4.5. <i>Desarrollo esbelto de software (DES)</i>	30
2.4.6. <i>Modelado ágil (MA)</i>	30
3. HERRAMIENTAS PARA LA GESTIÓN ÁGIL.	33
BIBLIOGRAFÍA.....	36

Índice de tabla

Tabla 1: Comparación de metodologías ágiles versus tradicionales.....	7
Tabla 3: Prácticas de la programación extrema	10

Índice de Ilustración

Ilustración 1: El ciclo de entrega en la programación extrema.	8
Ilustración 2: Las fases y la dinámica de la programación extrema.....	8
Ilustración 3: Tarjeta de historia para la descarga de documentos.....	11
Ilustración 4: Tarjetas de tareas para la descarga de documentos.	13
Ilustración 5: Descripción del caso de prueba para comprobar la validez de la tarjeta de crédito.....	14
Ilustración 6: El proceso de la programación extrema	16
Ilustración 7: Flujo del proceso Scrum	21
Ilustración 8: Desarrollo adaptativo de software	24
Ilustración 9: Desarrollo impulsado por las características [Coa99] (con permiso).....	29

UNIDAD IV METODOLOGÍAS ÁGILES

1. ¿QUÉ ES LA AGILIDAD?

Pero, ¿qué es la agilidad en el contexto de los trabajos de ingeniería de software? Ivar Jacobson [Jac02a] hace un análisis útil:

Agilidad es hoy la palabra mágica para describir un proceso de software moderno, todos son ágiles. Un equipo ágil, eficiente y capaz de responder adecuadamente a los cambios. El cambio es en gran parte software, hay cambios en el software. Miembros del grupo incorporados, debido a las nuevas tecnologías, de todo tipo y que tienen un efecto en el producto que se está creando o el proyecto que se está creando. Se debe indicar el apoyo para cambiar cualquier cosa que se haga en el programa; a veces se hace porque es corazón y alma de este. El equipo ágil entiende que el software es desarrollado por personas que trabajan en equipo, su capacidad y habilidad de colaboración es la base para el éxito del proyecto (Pressman, 2010).

En opinión de Jacobson, la propagación del cambio es el principal impulsor de la agilidad. Los ingenieros de software deben moverse rápidamente si quieren adaptarse a los rápidos cambios que describe Jacobson (Pressman, 2010).

Pero la resiliencia es más que una respuesta eficaz al cambio; sugerir estructuras de trabajo en equipo y actitudes que facilitan la comunicación (entre miembros del equipo, tecnólogos y empresarios, entre ingenieros de software y sus directivos, etc.); centrarse en la entrega rápida de programas de trabajo y no en productos de trabajo intermedios (no siempre buenos); integrar al cliente en el equipo de desarrollo e intentar deshacernos de la actitud de “nosotros y ellos” que aún prevalece en muchos proyectos de software; reconocer que planificar en un mundo incierto tiene sus límites y planificar el proyecto debe ser flexible (Pressman, 2010).

La agilidad se puede aplicar a cualquier proceso de software. Sin embargo, para lograr esto, es fundamental que se diseñe de forma que permita al equipo del proyecto adaptar las tareas y hacerlas directas, ejecute la planeación de una manera que comprenda el proceso del enfoque ágil del desarrollo, eliminando todos los productos de trabajo excepto los más importantes y manteniéndolos débiles, enfatizando la estrategia de distribución gradual que

hace trabajar al programa lo más rápido posible para el cliente, según el tipo de producto y el entorno de trabajar (Pressman, 2010).

1.1. ¿QUÉ ES UN PROCESO ÁGIL?

Cualquier proceso de software ágil se distingue por la forma en que se relacionan entre sí [Fow02] aborda suposiciones clave en la mayoría de los proyectos de software: (Pressman, 2010).

1. Es difícil predecir qué requisitos de software existirán y cuáles cambiarán. También es difícil predecir cómo y cuándo cambiarán las prioridades de los clientes de acuerdo al progreso del proyecto.
2. Para muchos tipos de software, el diseño y la construcción están estrechamente relacionados. Eso es que ambas operaciones deben realizarse simultáneamente, para que los modelos de los diseños se prueban inmediatamente después de la creación. Es difícil predecir cuántos diseños se requerirán antes de usar la compilación para probar el diseño.
3. Analizar, diseñar, construir y probar es tan impredecibles como desearíamos (desde el punto de vista de la planificación) (Pressman, 2010).

Con estos tres supuestos, surge una pregunta importante: ¿Cómo se crea un proceso que pueda manejar lo impredecible? La respuesta, como se ha dicho, está en la adaptabilidad del proceso (con el rápido desarrollo del proyecto y las condiciones técnicas). Por lo tanto, el proceso debe ser rápido y capaz de adaptarse. Pero la adaptación continua no logrará mucho sin progreso. Por lo tanto, el proceso de software ágil debe adaptarse gradualmente. Para lograr un ajuste gradual, un grupo de respuesta rápida a la demanda del cliente (de modo que sea posible hacer las adaptaciones apropiadas). Un catalizador efectivo para la retroalimentación de los clientes es un prototipo funcional o parte de un sistema operativo (Pressman, 2010).

Por lo tanto, debe instituirse una estrategia de desarrollo incremental. Deben entregarse incrementos de software (prototipos ejecutables o porciones de un sistema operativo) por un corto tiempo, para que la modificación vaya de la mano con el Cambio (inesperado). Este enfoque iterativo permite a los clientes evaluar regularmente las mejoras del software, proporcionar la retroalimentación necesaria al equipo de software y lograr un impacto en modificar el proceso para aprovechar los comentarios (Pressman, 2010).

1.2. PRINCIPIOS DE AGILIDAD

La Alianza Ágil (véase [Agi03]), [Fow01]) identifica 12 principios de agilidad para las personas que quieren lograrla:

1. La máxima prioridad es la satisfacción del cliente a través de una entrega rápida y continua de programas valiosos.
2. Las solicitudes de cambio son bienvenidas, incluso en una etapa avanzada de desarrollo. Los procesos ágiles aprovechan el cambio para obtener una ventaja competitiva del cliente.
3. Proporcionar programas funcionales regularmente, de dos semanas a varios meses, preferiblemente lo antes posible.
4. Las empresas y los desarrolladores deben trabajar juntos diariamente y de manera oportuna en todo el proyecto.
5. Los proyectos deben desarrollarse con personas motivadas. Debe brindarles el entorno y el apoyo que necesitan, y confiar en que harán el trabajo.
6. La forma más eficaz y eficiente de transmitir información a los miembros del equipo de desarrollo.
7. La principal medida de progreso es el programa que está funcionando.
8. Las operaciones ágiles promueven la sostenibilidad. Los patrocinadores, desarrolladores y usuarios deben poder mantener una velocidad constante indefinidamente.
9. La atención constante a la excelencia en ingeniería y el buen diseño mejora la agilidad.
10. La simplicidad es la clave: el arte de maximizar el trabajo inacabado.
11. Las mejores estructuras, requisitos y diseños provienen de sus equipos organizados.
12. El equipo piensa periódicamente en cómo ser más productivo y luego revisa y ajusta su comportamiento en consecuencia (Pressman, 2010).

No todos los modelos de procesos ágiles aplican estos 12 principios con la misma intensidad y algunas eligen Ignorar (o al menos soslayar) la importancia de uno o más de ellos. Sin embargo, los principios reconocen el espíritu de agilidad que se mantiene en cada uno de los modelos de procesos presentados (Pressman, 2010).

1.3. FACTORES HUMANOS

Los defensores del desarrollo de software ágil intentan subrayar la importancia del "factor individual". Como dijeron Cockburn y Highsmith [Coc01a]: "El desarrollo rápido se enfoca en el talento y las capacidades individuales, al tiempo que adapta los procesos para personas y grupos específicos". El punto principal de esta afirmación es el proceso de adaptación a las necesidades de las personas y de los grupos y no al revés (Pressman, 2010).

Si los miembros del equipo del programa van a crear las características del proceso que se aplicarán al desarrollo del programa, entonces debe haber un cierto número de las principales características que tienen que compartir los equipos ágiles, entre ellas tenemos las siguientes: (Pressman, 2010).

Competencia. En el contexto del desarrollo rápido (así como en la ingeniería de software), las "habilidades" incluyen talentos innatos y habilidades específicas relacionadas con Software y conocimiento general del proceso que el equipo eligió adoptar. La eficiencia del proceso y el conocimiento pueden y deben ser considerados para todos los miembros ágiles del equipo (Pressman, 2010).

Enfoque Común. Como los miembros ágiles del equipo realizan diferentes tareas y aportan diferentes conjuntos de habilidades al proyecto, todos deben centrarse en un objetivo: la entrega al cliente de un incremento de software que funcione, dentro de la fecha prometida. Para lograrlo, el grupo también destacará en adaptaciones continuas (pequeñas y grandes) que hagan un proceso a la medida de las necesidades del grupo (Pressman, 2010).

Colaboración. La ingeniería de software (independientemente del proceso) se ocupa de la evaluación, el análisis y el uso de la información que se envía al equipo de software; la generación de información permite ayudar a todos los participantes a comprender el trabajo del equipo; y generar información (software informático y bases de datos relacionadas) que proporcionan a los clientes valor comercial. Para realizar estas tareas, los miembros del equipo deben colaborar entre sí y con todos los asistentes (Pressman, 2010).

Habilidad para tomar decisiones. Cualquier buen equipo de software (incluso equipo ágil) debe ser libre de controlar su propio destino. Esto implica dar autonomía al grupo: el derecho a tomar decisiones técnicas y comerciales el proyecto (Pressman, 2010).

Capacidad para resolver problemas difusos. Los administradores de software deben darse cuenta de que los equipos ágiles tendrán que lidiar constantemente con la ambigüedad y que

será sacudido de manera permanente por el cambio. En algunos casos, el grupo debe aceptar el hecho de que el problema que están resolviendo ahora puede no ser el problema a resolver mañana. Sin embargo, las lecciones aprendidas de cualquier actividad que involucre la resolución de problemas (incluso uno que resuelva el problema equivocado) beneficiará al equipo más adelante en el proyecto (Pressman, 2010).

Confianza mutua y respeto. El equipo ágil debería convertirse en lo que DeMarco y Lister [DeM98] llaman “pegado”. Un grupo muy unido tiene confianza y el respeto necesario para hacer “su tejido tan fuerte que el todo es más que la suma de sus partes” [DeM98] (Pressman, 2010).

Organización propia. En el contexto del desarrollo ágil, la autorregulación incluye tres cosas:

- 1) el equipo ágil se organiza para hacer el trabajo,
- 2) el equipo organiza el proceso que mejor se adapte a su entorno local,

3) el equipo organiza el horario de trabajo para distribuir el crecimiento gana el mejor azúcar posible del programa. Sin embargo, la autorregulación tiene algunas ventajas técnicas pero lo más importante es que mejora la cooperación y aumenta la moral del equipo, esencialmente, el equipo actúa como su propio gerente. Ken Schwaber [Sch02] aborda estos temas cuando escribe: “El equipo selecciona cuánto trabajo cree que puede realizar en cada iteración, y se compromete con la labor, nada motiva a un grupo como que una persona comprometida con nada motiva a un equipo como aceptar la responsabilidad de los compromisos que han hecho (Pressman, 2010).

2. METODOLOGÍAS.

Hasta el momento se han considerado modelos de desarrollo de software, como abstracciones del proceso, las metodologías representan otro paso más porque no solo se enfocan en describir las etapas del proceso, sino también muestran cómo hacerlo como gestionar los documentos, procedimientos, técnicas y herramientas auxiliares (Patricia et al., 2018).

Sommerville (2005) resume el término 'metodología' en el desarrollo de software como: Un “enfoque estructurado” destinado a facilitar la producción de software de alta calidad a un costo razonable. Se ocupa de: procesos, modelos, tareas, procedimientos y herramientas.

En términos de metodología, se debe hacer una distinción entre métodos tradicionales, formales o "primer enfoque estructurado" y el "ágiles" que surgen en contraposición a las primeras. Los enfoques tradicionales han evolucionado desde la década de 1980, se han mostrado varios modelos (Patricia et al., 2018).

Según Pérez (2011), los marcos tradicionales imponen una estricta planificación un proyecto minucioso, respaldado por herramientas y una gran carga de trabajo en planificación, diseño y documentación, con el objetivo de hacer predecibles los desarrollos en el marco de tiempo y costo; Recuerda que se deriva de la resolución de problemas (Patricia et al., 2018).

Experimentó una crisis de software hace décadas, los más comunes y citados en los textos son: Desarrollo de Aplicaciones (RAD), Proceso Racional Unificado (RUP) y Soluciones Microsoft marco (MSF) [Rapid Application Development (RAD), Rational Unified Process (RUP) y Microsoft Solution Framework (MSF)] (Patricia et al., 2018).

Implementando metodologías clásicas en proyectos de mediana y gran escala se cumplen los requisitos de tiempo de respuesta, las solicitudes son incorrectas y se han modificado los resultados ineficaces porque se dedicó más tiempo a diseñar y dar respuesta de control a cualquier cambio en las especificaciones; esto no es compatible con el método de realización de análisis y documentación, haciendo el desarrollo de software es un proceso ineficiente e ineficiente. (Pérez, 2011) (Patricia et al., 2018).

Aparecen en respuesta las metodologías ágiles para el desarrollo de software, alternativas que buscan enfocarse en el software más que en la arquitectura o documentación, con un enfoque iterativo, respondiendo a requisitos cambiantes y entregar trabajos desde las primeras etapas a través del compromiso del cliente. La buena diferencia entre las metodologías tradicionales y las llamadas rápidas es presentada en la Tabla 1, compilada originalmente por Canós et al. (2003) y ampliado por Ceniza (2014) (Patricia et al., 2018).

Metodologías ágiles	Metodologías tradicionales
Se basan en heurísticas provenientes de prácticas de producción de código	Se basan en normas provenientes de estándares seguidos por el entorno de desarrollo
Énfasis en los aspectos humanos: el individuo y el trabajo en equipo.	Énfasis en la definición del proceso: roles, actividades y artefactos.
Preparados para cambios durante el proyecto	Cierta resistencia a los cambios
Impuestas internamente por el equipo	Impuestas externamente
Proceso menos controlado, con pocos principios	Proceso muy controlado, numerosas normas
Contrato flexible e incluso inexistente	Contrato prefijado
El cliente es parte del desarrollo	Cliente interactúa con el equipo de desarrollo mediante reuniones
Grupos pequeños (<10) con pocos roles, más genéricos y flexibles.	Grupos grandes con Más roles y más específicos.
Orientada a proyectos pequeños, y en el mismo lugar.	Aplicables a proyectos de cualquier tamaño, pero suelen ser especialmente efectivas/usadas en proyectos grandes.
Pocos artefactos El modelo es prescindible, modelos desechables.	Más artefactos. El modelo es esencial, mantenimiento en los modelos.
Menor énfasis en la arquitectura del software, se va definiendo y mejorando a lo largo del proyecto.	La arquitectura del software es esencial, se define tempranamente en el proyecto.

Tabla 1: Comparación de metodologías ágiles versus tradicionales

Fuente: Canós et al. (2005); Cendejas (2014) y (Patricia et al., 2018)

2.1. PROGRAMACIÓN EXTREMA (XP).

La Programación Extrema (XP) es probablemente el método ágil más conocido y más utilizado. (Beck, 2000) aplicó el nombre porque el método se desarrolló utilizando las mejores prácticas reconocidas, como el desarrollo iterativo y la participación de clientes "extremos" (Sommerville, 2005).

En la programación extrema, todos los requisitos se expresan como escenarios (llamados historias de usuario), que se ejecutan directamente como una serie de tareas. Los programadores actúan en parejas y desarrollan pruebas para cada tarea antes de codificar (Sommerville, 2005).

Todas las pruebas se ejecutarán con éxito cuando el código nuevo se integre en el sistema. Hay un pequeño lapso de tiempo entre las distribuciones del sistema. La ilustración 1, muestra el proceso XP para generar incrementos en el sistema en ejecución (Sommerville, 2005).

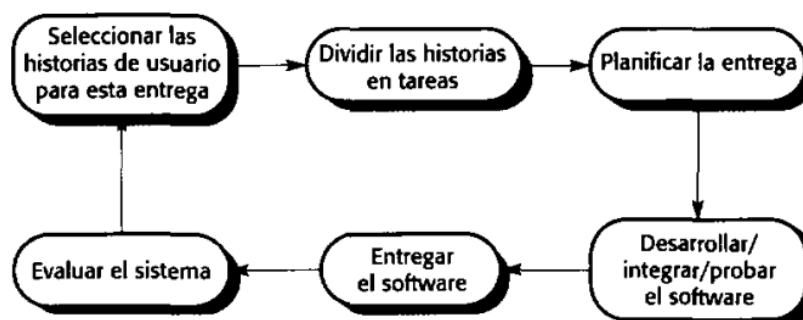


Ilustración 1: El ciclo de entrega en la programación extrema.

Fuente: (Sommerville, 2005)

La programación extrema (XP) es un método para equipos pequeños y medianos. XP Proporciona 12 prácticas (regla) con la finalidad de reducir los costos del cambio. LA programación extrema es la aceptación de las mejores metodologías de desarrollo y ciclo de vida y aplicarlo con flexibilidad, según los requisitos y características de cada parte (componentes, funciones, fase, etc.) del proyecto en desarrollo (M. Gómez et al., 2019).

En otras palabras, la programación extrema sugiere no adoptar una sola metodología o ciclo de vida de desarrollo del programa durante el desarrollo del proyecto del programa, sino que el modelo de desarrollo propuesto cambia dinámicamente dependiendo de las características del proyecto. La ilustración 2 muestra las etapas y la dinámica de programación extrema (M. Gómez et al., 2019).

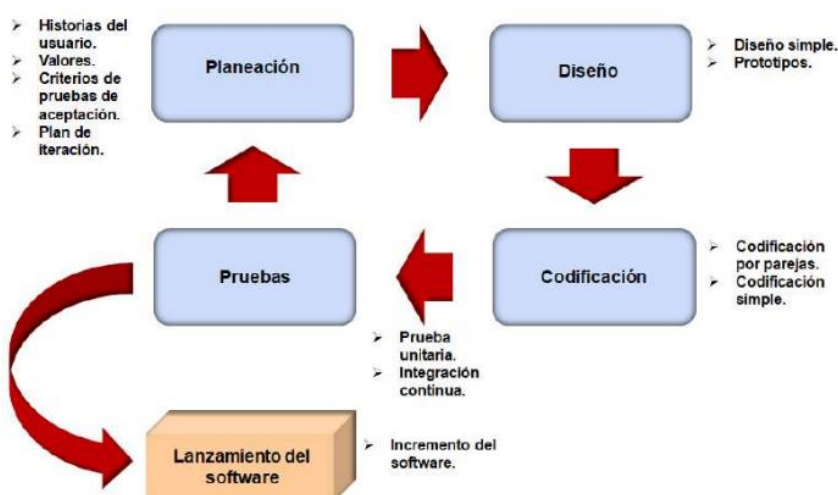


Ilustración 2: Las fases y la dinámica de la programación extrema

Fuente: (M. Gómez et al., 2019).

El poder de la programación extrema no es el resultado de las 12 práctica, pero las características emergentes surgen de su combinación (sinergia). Estos estándares están destinados a producir diseños y pruebas simples, sistema automático y posición de diseño de mejora continua. (M. Gómez et al., 2019).

La programación extrema incluye varias prácticas, resumidas en la tabla 2, que se adhieren a los principios del estilo ágil: (Sommerville, 2005)

1. Se logra un mayor desarrollo a través de versiones menores frecuentes del sistema y un enfoque basado en datos para la especificación de requisitos basados en historias o escenarios de los clientes que se pueden utilizar como base para el proceso de planificación (Sommerville, 2005).

2. El compromiso del cliente requiere un compromiso de tiempo completo de los clientes en el equipo de desarrollo. Los representantes de los clientes participan en desarrollar y asumir la responsabilidad de definir las pruebas de aceptación del sistema (Sommerville, 2005).

3. El interés en las personas en vez de en los procesos, se implementa a través de la programación en pares y la propiedad colectiva del sistema y el código del sistema y un proceso del desarrollo sostenible no implica jornadas laborales excesivas (Sommerville, 2005).

4. El cambio ocurre a través de transformaciones regulares en los lanzamientos del sistema, integración continua y preprobada (Sommerville, 2005).

5. La simplicidad se mantiene mediante la refactorización constante para mejorar la calidad del código y el uso de diseños simples que no anticipan cambios futuros en el sistema (Sommerville, 2005).

Principio o práctica	Descripción
Planificación incremental	Los requerimientos se registran en tarjetas de historias y las historias a incluir en una entrega se determinan según el tiempo disponible y su prioridad relativa. Los desarrolladores dividen estas Historias en <Tareas> de desarrollo.
Entregas pequeñas	El mínimo conjunto 001 de funcionalidad que proporcione valor de negocio se desarrolla primero. Las entregas del sistema son frecuentes e incrementalmente añaden funcionalidad a la primera entrega.

Diseño sencillo	Solo se lleva a cabo el diseño necesario para cumplir los requerimientos actuales.
Desarrollo previamente probado	Se utiliza un sistema de pruebas de unidad automatizado para escribir pruebas para nuevas funcionalidades antes de que éstas se implementen.
Refactorización	Se espera que todos los desarrolladores refactoricen el código continuamente tan pronto como encuentren posibles mejoras en el código. Esto conserva el código sencillo y mantenible.
Programación en parejas	Los desarrolladores trabajan en parejas, verificando cada uno, el trabajo del otro y proporcionando la ayuda necesaria para hacer siempre un buen trabajo.
Propiedad colectiva	Las parejas de desarrolladores trabajan en todas las áreas del sistema, de modo que no desarrollen islas de conocimientos y todos los desarrolladores posean todo el código. Cualquiera puede cambiar cualquier cosa.
Integración continua	En cuanto acaba el trabajo en una tarea, se integra en el sistema entero. Después de la integración, se deben pasar al sistema todas las pruebas de unidad.
Ritmo sostenible	No se consideran aceptables grandes cantidades de horas extras, ya que a menudo el efecto que tienen es que se reduce la calidad del código y la productividad a medio plazo.
Cliente presente	Debe estar disponible al equipo de la XP un representante de los usuarios finales del sistema (el cliente) a tiempo completo. En un proceso de la programación extrema, el cliente es miembro del equipo de desarrollo y es responsable de formular al equipo los requerimientos del sistema para su implementación

Tabla 2: Prácticas de la programación extrema

Fuente: (Sommerville, 2005)

En el proceso XP, el cliente está muy involucrado en las especificaciones y requisitos del sistema de prioridad. Los requisitos no son especificados como una lista de las funciones requeridas del sistema. En su lugar, los clientes del sistema pasan a formar parte del equipo de desarrollo y analizan escenarios con otros miembros del equipo. Juntos, desarrollaron una «tarjeta de historias» (story card) para captar las necesidades del cliente. Luego, el equipo de desarrollo intentará implementar este escenario en una entrega futura del programa. En la ilustración 3 se muestra un ejemplo de una tarjeta de historia para el sistema LIBSYS (Sommerville, 2005).

Descarga e impresión de un artículo

En primer lugar, seleccione el artículo que desea de una lista visualizada. Tiene entonces que decirle al sistema cómo lo pagará —se puede hacer a través de una suscripción, una cuenta de empresa o mediante una tarjeta de crédito.

Después de esto, obtiene un formulario de derechos de autor del sistema para que lo rellene. Cuando lo haya enviado, se descarga el artículo en su computadora.

Elija una impresora y se imprimirá una copia del artículo. Le dice al sistema que la impresión se ha realizado correctamente.

Si es un artículo de sólo impresión, no puede guardar la versión en PDF, por lo que automáticamente se elimina de su computadora.

Ilustración 3: Tarjeta de historia para la descarga de documentos

Fuente: (Sommerville, 2005).

Una vez desarrolladas las tarjetas de historias, el equipo de desarrollo las divide en tareas y estima el esfuerzo y los recursos necesarios para implementarlas. El cliente entonces priorizará las historias a implementar, seleccionando las historias que se pueden utilizar de inmediato para proporcionar un apoyo comercial útil. Seguramente, cuando cambia el orden, las historias incompletas también cambian o pueden ser eliminadas (Sommerville, 2005).

Si se requiere una modificación en un sistema entregado se desarrollan nuevas tarjetas de historia y, nuevamente, depende del cliente decidir si estos cambios tienen prioridad sobre las nuevas funcionalidades. La programación extrema adopta un enfoque "extremo" para el desarrollo iterativo, se pueden crear nuevas versiones del programa varias veces al día, y los complementos se entregan a los clientes aproximadamente cada dos meses. Cuando el programador construye el sistema para crear una nueva versión, debe ejecutar todas las pruebas automatizadas existentes además de probar nuevas funciones (Sommerville, 2005).

El programa solo se acepta si se pasan todas las pruebas. El principio básico de la ingeniería de software tradicional es que el software debe diseñarse para cambiar. En otras palabras, debe anticipar cambios futuros en el software y diseñar en consecuencia así que estos cambios se pueden hacer fácilmente. Sin embargo, la programación extrema rechaza este principio sobre la base del hecho de que el diseño para el cambio es a menudo un intento inútil. Los cambios esperados a menudo nunca se realizan, y ya se están realizando solicitudes de cambios radicalmente diferentes (Sommerville, 2005).

El problema de hacer cambios no planificados es que tienden a degradar la calidad de la Ingeniería de software, cada vez es más difícil implementar cambios. La programación extrema resuelve este problema al sugerir que el programa debe reestructurarse constantemente. Esto significa que el equipo de programación está buscando posibles mejoras del software y publicarlo inmediatamente. Por lo tanto, el software siempre debe ser fácil de entender y cambiar a medida que se publican nuevas historias (Sommerville, 2005).

2.1.1. Pruebas en XP

Una de las principales diferencias entre el desarrollo iterativo y la planificación impulsada por el desarrollo se encuentra el método de verificación del sistema. Con el desarrollo iterativo, no existe una especificación del sistema que un equipo de prueba externo pueda usar para desarrollar pruebas del sistema. Por lo tanto, algunos enfoques de desarrollo iterativo tienen un proceso de prueba muy complejo e informal. Para evitar algunos problemas de prueba y autenticación del sistema XP hace más énfasis en el proceso de prueba en comparación con otros métodos ágiles (Sommerville, 2005).

Las verificaciones del sistema son fundamentales en XP, donde se desarrolló un enfoque para reducir la probabilidad de crea nuevos incrementos en el sistema que introducen errores en los programas existentes (Sommerville, 2005).

Las principales características de la prueba en XP son:

1. Desarrollo probado.
2. Desarrollar pruebas adicionales de escenarios.
3. Participación de los usuarios en el desarrollo y validación de las pruebas.
4. Utilización de bancos de pruebas automatizados.

Una de las innovaciones más importantes de XP es el desarrollo de pruebas previas, al escribir primero las pruebas, define completamente la interfaz y las especificaciones de rendimiento para el trabajo que se va a desarrollar. Los problemas se mitigan en requisitos de interfaz y confusión de la interfaz. Este enfoque se puede aplicar en cualquier proceso en el que existe una clara relación entre los requisitos del sistema y el código que envíe ese

requerimiento. En XP todavía puedes ver este enlace porque las tarjetas de historia representan órdenes desglosadas en tareas, y las tareas son las unidades principales de implementación (Sommerville, 2005).

Como se señaló, los requisitos del usuario en XP se expresan como actitudes o historias y los usuarios priorizan el crecimiento. El equipo de desarrollo evalúa cada escenario y lo divide en tareas. Cada tarea representa una característica diferente del sistema y luego se puede diseñar una prueba unitaria para esa tarea. Por ejemplo, la ilustración 4 muestra algunas tarjetas de presentación desarrolladas desde la página de registro para cargar documentos ilustración 3 (Sommerville, 2005).

Cada tarea crea una o más pruebas unitarias para verificar la implementación descrita en esta tarea. Por ejemplo, la ilustración 5 es una breve descripción de un caso de prueba que se ha desarrollado para verificar que se haya publicado un número de tarjeta de crédito válido (Sommerville, 2005).

El rol del cliente de prueba es ayudar a desarrollar pruebas de aceptación para las historias que se implementarán en la próxima versión del sistema, la prueba de aceptación es el proceso en el que se prueba el sistema utilizando los datos de los clientes para verificar que cumple con sus necesidades reales (Sommerville, 2005).

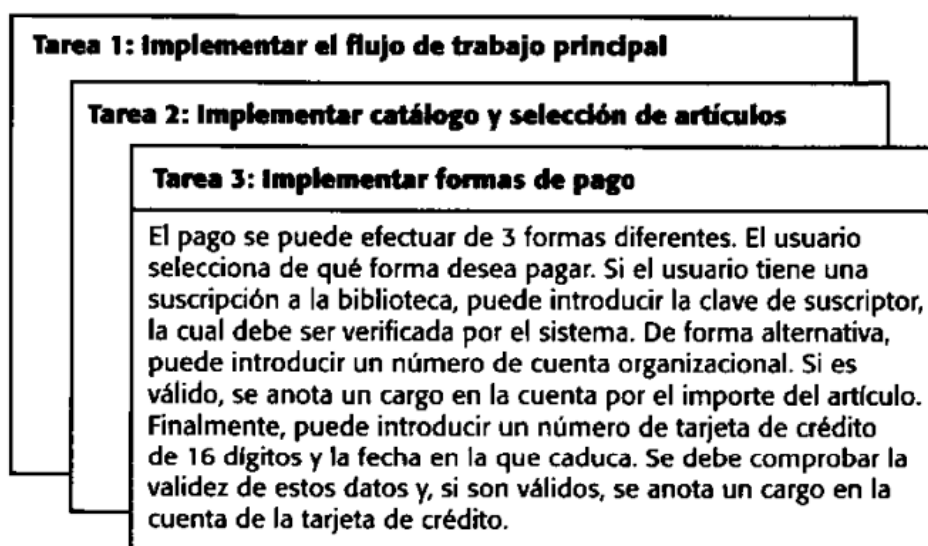


Ilustración 4: Tarjetas de tareas para la descarga de documentos.

Fuente: (Sommerville, 2005).

Prueba 4: Prueba de la validez de la tarjeta de crédito

Entrada:

Una cadena que representa el número de tarjeta de crédito y dos enteros que representan el mes y el año de la caducidad de la tarjeta.

Pruebas:

Comprobar que todos los bytes de la cadena son dígitos.

Comprobar que el mes se encuentra entre 1 y 12 y que el año es mayor o igual que el año actual.

Utilizando los 4 primeros dígitos del número de tarjeta de crédito, comprobar que el emisor de la tarjeta es válido consultando la tabla de emisores de tarjetas. Comprobar la validez de la tarjeta de crédito enviando el número de tarjeta y la fecha en la que caduca el emisor de la tarjeta.

Salida:

OK o un mensaje de error indicando que la tarjeta no es válida.

Ilustración 5: Descripción del caso de prueba para comprobar la validez de la tarjeta de crédito.

Fuente: (Sommerville, 2005).

En XP, las pruebas de aceptación, como el desarrollo, son incrementales, para esta historia en particular, la prueba de admisión consistirá en seleccionar y pagar algunos documentos de diferentes maneras e impresas en diferentes impresoras. De hecho, podría ser que desarrollará una serie de pruebas de aceptación en lugar de una sola (Sommerville, 2005).

El desarrollo y uso de bancos de prueba automatizados previamente probados son las principales ventajas del método XP. En lugar de escribir el código del programa y luego las pruebas para este código, el desarrollo probado previamente significa que las pruebas se escriben antes que el código. Básicamente, las pruebas se escriben como un componente que se ejecuta antes de que se ejecute la tarea (Sommerville, 2005).

Una vez que se implementa el software, las pruebas se pueden ejecutar de inmediato. Este componente de prueba debe ser una aplicación independiente que tiene que simular el envío de entrada para ser probado y debe verificar que los resultados coinciden con las especificaciones de salida. Un conjunto de pruebas automatizadas es un sistema que envía estas pruebas automatizadas al tiempo de ejecución. Con el desarrollo probado previamente, siempre hay un conjunto de pruebas que se pueden ejecutar rápida y fácilmente. Esto significa que cada vez que agrega funcionalidades del sistema, puede ejecutar pruebas y detectar inmediatamente los problemas introducidos por el nuevo código (Sommerville, 2005).

Durante el desarrollo prevalidado, los responsables de las tareas necesitan entender completamente las especificaciones, para que puedan escribir las pruebas del sistema. Esto significa que deben aclararse las ambigüedades y omisiones en las especificaciones antes de iniciar la implementación. Además, también previene el problema del «retraso de las pruebas», porque los desarrolladores de sistemas trabajan a mayor velocidad que los probadores (Sommerville, 2005).

Realice una prueba previa de las aplicaciones y tienda a omitir las pruebas para el mantenimiento de horario especificado. Sin embargo, el desarrollo probado previamente no siempre funciona como se esperaba, los programadores prefieren programar sobre pruebas y, a veces, escribir pruebas incompletas sin marcar posiciones excepcionales. Puede ser difícil escribir algunas pruebas. Por ejemplo, en una interfaz de usuario compleja, muchas veces es difícil escribir pruebas unitarias para el código que implementa "vistas lógicas" y flujos de trabajo entre pantallas. Al final, es difícil juzgar qué tan completo es el grupo de una prueba, incluso si tiene una gran cantidad de pruebas del sistema, es posible que las pruebas no brinden una cobertura completa. Es posible que no se ejecuten o no funcionen partes cruciales del sistema y por lo tanto, se queden sin probar (Sommerville, 2005).

Confiar en los clientes para ayudar a desarrollar pruebas de aceptación es a veces un problema grave durante la prueba de XP. Las personas en el rol de cliente tienen muy poco tiempo y es posible que no trabajen a tiempo completo con el equipo de desarrollo. Los clientes pueden pensar que proporcionar requisitos es una contribución adecuada y puede mostrarse renuente a participar en el proceso de prueba (Sommerville, 2005).

2.1.2. El Proceso Xp

La programación extrema usa un enfoque orientado a objetos, como su modelo de desarrollo preferido e incluye un conjunto de reglas y prácticas que ocurren en el contexto de cuatro actividades estructurales: planeación, diseño, codificación y pruebas. La ilustración 6, muestra el proceso de XP y destaca algunas de las ideas y tareas clave asociadas con cada proceso estructural. Los siguientes párrafos resumen las principales actividades de XP (Pressman, 2010).

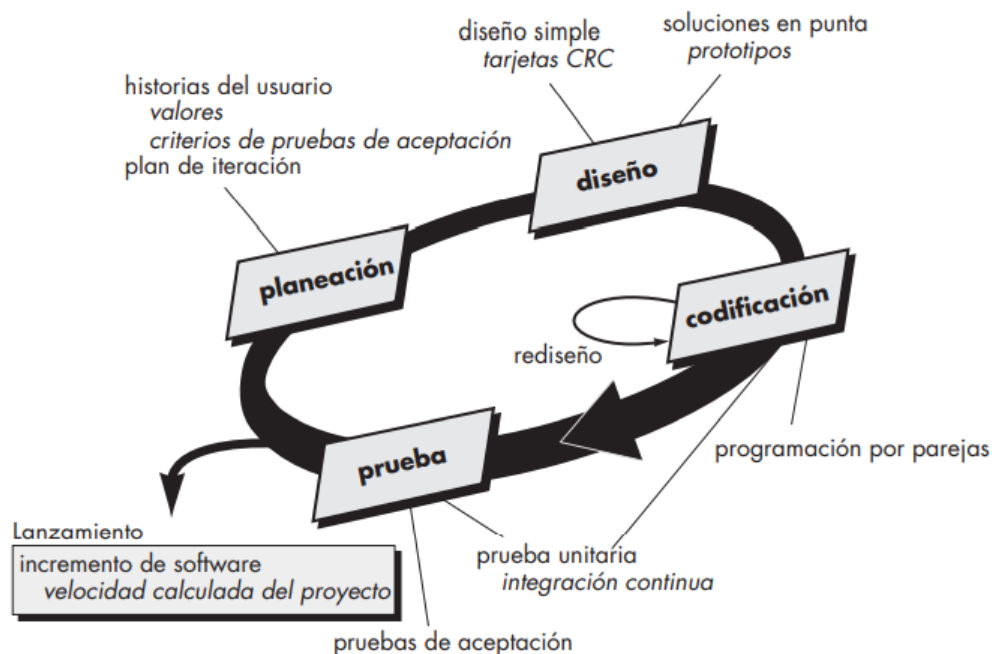


Ilustración 6: El proceso de la programación extrema

Fuente: (Pressman, 2010).

Planeación Una actividad de planificación (también conocida como juego de planeación) comienza con escuchar, una actividad que reúne los requisitos que permiten a los miembros técnicos de un equipo hacerlo (Pressman, 2010).

El equipo XP entiende el contexto del programa y es sensible la Salida y principales características y funciones requerida. Escuchar crea ciertas "historias" (también conocidas como historias de usuarios) que describen el resultado, las características requeridas y la funcionalidad del software creado. Cada historia (similar a los casos de uso descritos en el capítulo 5) escritos por el cliente y colocados en la tarjeta indexado. El cliente asigna un valor (es decir, prioridad) a la historia con base en el valor general de la característica o función para el negocio, luego, los miembros del equipo de XP evalúan cada historia y determinan un costo medido en semanas de desarrollo, si alguien aprecia que la historia tarda más de tres semanas en desarrollarse y se le pide al cliente que divida en las historias pequeñas nuevamente el valor y el costo. Es importante señalar que siempre es posible escribir nuevas historias (Pressman, 2010).

Los clientes y los desarrolladores trabajan juntos para decidir cómo armar las historias en las próximas versiones (siguiente incremento de software) que será desarrollada por el

equipo de XP. Una vez que el compromiso se haga en el momento de la entrega (acordar qué historias incluir, fecha, distribución y otros aspectos del proyecto), el equipo de XP ordena las historias q serán el desarrolladas en tres maneras: 1) Todas las historias se publicarán inmediatamente (dentro de pocas semana), 2) las historias más valiosas se incluirán en el cronograma de actividades y se ejecutarán primero, o 3) las historias más peligrosas serán parte del cronograma de actividades y las actividades se implementarán primero (Pressman, 2010).

Después del primer lanzamiento del proyecto (también conocido como complementos), el equipo de XP calcula la velocidad de esto. En pocas palabras, la velocidad del proyecto es el número de historias de los clientes generados en la primera entrega (Pressman, 2010).

Las velocidades del proyecto se utilizan para: 1) ayudar a estimar las fechas de entrega y planificar las operaciones para hacer un seguimiento de las entregas, y 2) determinar si se han realizado o no compromisos significativos para todas las historias durante todo el período de desarrollo del proyecto. Si esto sucede, el contenido de la entrega cambiará o se cambia la fecha de entrega final. A medida que avanza el trabajo, los clientes pueden agregar historias y cambiar el valor de una ya existente, descomponerlas o eliminarlas. El equipo de XP luego revisará las entregas faltantes y cambiará sus planes en consecuencia (Pressman, 2010).

Diseño. El diseño de XP sigue estrictamente el principio MS (mantenlo sencillo). un diseño

Siempre se prefiere una representación simple a una representación más compleja. Además del diseño guía la implementación de una historia como está escrita: ni más, ni menos. Es frustrante diseñar características adicionales porque el desarrollador piensa que serán necesarias más adelante (Pressman, 2010).

XP promueve el uso de tarjetas CRC, como un mecanismo eficaz para pensar en los programas en un contexto orientado a objetos. El mapa CRC (clase-responsabilidad-colaborador) define y organiza clases relacionadas orientadas a objetos para aumentar el programa actual. La tarjeta CRC es el único producto del trabajo de diseño que se creó como parte del proceso XP. Si el problema de diseño es difícil de resolver al diseñar una historia, XP recomienda crear inmediatamente un prototipo de trabajo de esta parte del diseño (Pressman, 2010).

Luego se implementa y evalúa el prototipo de diseño, conocido como solución en punta, el objetivo es reducir los riesgos de iniciar la implementación real y validar las estimaciones iniciales para la historia que contiene problemas de diseño. Se dijo anteriormente que XP fomenta el rediseño, que es una técnica de construcción que también es una forma de mejorar el diseño. Fowler [Fow00] ¹describe el rediseño del modo siguiente:

“Rediseño es el proceso mediante el cual se cambia un sistema de software en forma tal que no altere el comportamiento externo del código, pero sí mejore la estructura interna. Es una manera disciplinada de limpiar el código [y modificar o simplificar el diseño interno] que minimiza la probabilidad de introducir errores. En esencia, cuando se rediseña, se mejora el diseño del código después de haber sido escrito”.

Dado que el diseño de XP virtualmente no utiliza notación y genera pocos, si alguno, productos del trabajo que no sean tarjetas CRC y soluciones en punta, el diseño es un artefacto en transiciones que pueden y deben modificarse constantemente a medida que avanza la construcción (Pressman, 2010).

El propósito del rediseño es controlar estos cambios y recomendar pequeños cambios en el diseño que "son capaces de mejorarlo en forma radical" [Fow00]². Sin embargo, cabe señalar que el esfuerzo requerido para el rediseño aumenta exponencialmente con el tamaño de la aplicación. El concepto básico en XP es que el diseño ocurre antes y después de que comience la codificación. Rediseño significa que el diseño se realiza continuamente mientras se construye el sistema. De hecho, el diseño en sí le dará al equipo XP una guía de mejora del diseño (Pressman, 2010).

Codificación También se desarrollan historias y trabajos de diseño inicial, el equipo no comienza a codificar, sino que desarrolla una serie de pruebas a cada historia que se va a incluir en la entrega de la versión actual (incremento de software). Una vez que se crean las pruebas unitarias, los desarrolladores están mejor capacitados en lo que se necesita hacer para pasar la prueba. No se agrega nada extraño (MS). Una vez que el código está completo, se le aplican pruebas unitarias inmediatamente, con lo que se obtenga de los comentarios instantáneos para los desarrolladores (Pressman, 2010).

Un concepto clave a la hora de realizar operaciones de codificación (y uno de los aspectos del que más se habla en la XP) es programación en pareja. XP recomienda que dos personas

¹ Fowler, M., et al., Refactoring: Improving the Design of Existing Code, Addison-Wesley, 2000.

² Fowler, M., et al., Refactoring: Improving the Design of Existing Code, Addison-Wesley, 2000

trabajen juntas en la estación de trabajo para crear un código de historia. Esto proporciona un mecanismo para resolver problemas en tiempo real (es frecuente que dos cabezas piensen más que una) y para el aseguramiento de la calidad también en tiempo real (el código se revisa cuando se genera). También permite a los desarrolladores centrarse en el problema a tratar, de hecho, cada uno juega un papel ligeramente diferente. Por ejemplo, uno de ellos probablemente piense en los detalles del código de una parte particular del diseño, mientras que la otra es asegúrese de seguir los estándares de codificación (parte requerida de XP) o de que el código de la historia satisfará la prueba unitaria que se desarrolla para validar el código contra la historia. Cuando los pares de programadores completan su trabajo, el código que desarrollan debe integrarse con el trabajo de los demás. En algunos casos, esto se hace diariamente por un grupo de integración. En otros casos, la integración es entre parejas de programadores. La integración continua evita problemas de compatibilidad e interfaces, y proporciona un entorno "a prueba de humo" (consulte el Capítulo 17) que ayuda a detectar errores antes (Pressman, 2010).

Prueba: Se ha dicho que crear pruebas unitarias antes de comenzar a codificar es el elemento básico del método XP. Las pruebas unitarias generadas deben implementarse con una arquitectura que permita automatizarlas (para que puedan ejecutarse muchas veces y fácilmente). Esto motiva la estrategia de prueba de regresión cada vez que se cambia el código (esto sucede a menudo, según la filosofía de rediseño en XP). Dado que las pruebas de unidades individuales se organizan en un "conjunto de pruebas universal" [Wel99], las pruebas y la validación de la integridad del sistema se pueden realizar a diario (Pressman, 2010).

Esto le da al equipo XP un indicador consistente de progreso y también da señales de alerta si las cosas no van bien. Wells [Wel99] ³ dice: "Resuelva pequeños problemas cada poco minuto, ahora, corregir pequeños problemas cada cierto número de horas lleva menos tiempo que resolver grandes problemas desde el principio" (Pressman, 2010).

La prueba de aceptación de XP, también conocida como prueba de cliente, está determinada por los clientes que se centran en las características y funciones generales del sistema visual y puede ser revisado por los clientes. Las pruebas de admisión tienen sus raíces

³ Wells, D., "XP—Unit Tests", 1999, disponible en www.extremeprogramming.org/rules/unittests.html.

en la historia de los usuarios se publican como parte de la versión del software (Pressman, 2010).



Recuerde que. Las pruebas de aceptación se derivan de las historias de los usuarios.

2.2. SCRUM

Scrum (el nombre derivado de un juego específico que tiene lugar durante un partido de rugby) es un método ágil de desarrollo de software ideado por Jeff Sutherland y su equipo de desarrollo a principios de la década de 1990. En los últimos años, Schwaber y Biddle [Sch01a] han desarrollado métodos Scrum. Los principios de Scrum están alineados con la declaración ágil y se utilizan para guiar las actividades de desarrollo a través de un proceso de análisis que incluye las siguientes actividades estructurales: requerimientos, análisis, diseño, evolución y entrega (Pressman, 2010).

En cada actividad estructural, las tareas ocurren con el modelo de proceso llamado sprint. Trabajo realizado en un sprint (la cantidad que requiere cada actividad estructural variará según la complejidad y el tamaño del producto) adaptado al problema específico en cuestión y, a menudo modificado, en tiempo real por el Equipo Scrum (Pressman, 2010).

El proceso general del proceso Scrum se muestra en la ilustración 7. Scrum enfatiza el uso de un conjunto de patrones de proceso del software [Noy02] con eficacia comprobada en plazos extremadamente ajustados, requerimientos cambiantes y negocios críticos. Cada uno de estos modelos de procesos define un conjunto de acciones de desarrollo: (Pressman, 2010) (Pressman, 2010).

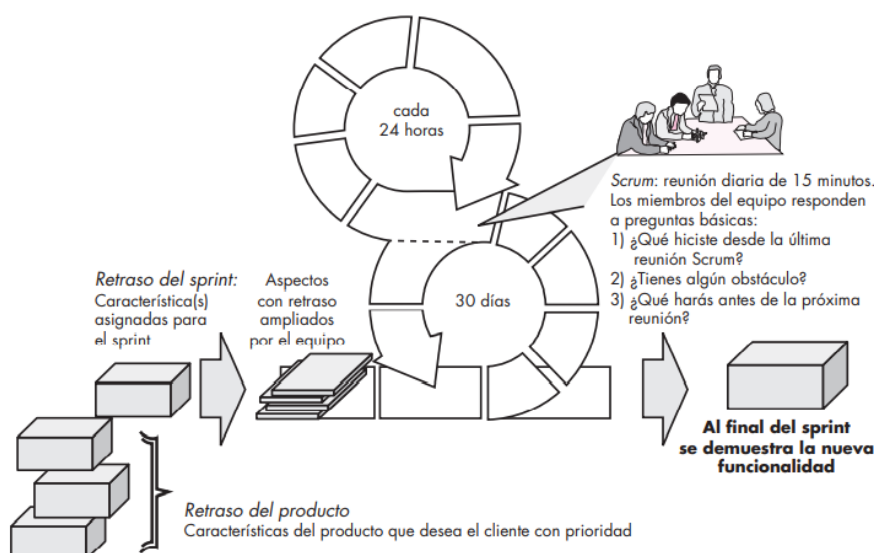


Ilustración 7: Flujo del proceso Scrum

Fuente: (Pressman, 2010).

Retraso: lista de prioridades de los requisitos del proyecto o entregables del proyecto que dan al cliente un valor del negocio. Se pueden agregar otros aspectos al retraso (así es como se introducen los cambios). Revisión del gerente de proyecto evalúa el retraso y actualiza las prioridades según sea necesario (Pressman, 2010).

Sprint: incluye las unidades de negocio necesarias para completar un requisito en el tiempo establecido, debe establecerse para un período preestablecido (generalmente 30 días). Durante el sprint no se introducen cambios (por ejemplo, aspectos de trabajo retrasado). Por lo tanto, los sprints permiten que los miembros del equipo trabajen en un entorno a corto plazo, pero es estable (Pressman, 2010).

Reuniones Scrum: Son reuniones breves (normalmente de 15 minutos) en las que el equipo Scrum ejecuta todos los días. Hay tres preguntas principales que todos los miembros del equipo deben responder [Noy02]: (Pressman, 2010).

- ¿Qué has hecho desde la última reunión del equipo?
- ¿Cuáles son los obstáculos que enfrenta?
- ¿Qué planea hacer hasta la próxima reunión del equipo?

El líder del equipo, conocido como maestro Scrum, facilita la reunión evalúa las respuestas de cada persona. La junta Scrum ayudan al equipo a detectar problemas potenciales a medida que surgen, asimismo, estos encuentros cotidianos conducen a la “socialización del

conocimiento” [Bee99], En cualquier estructura de grupo se promueve una estructura de equipo con la organización propia (Pressman, 2010).

Demostraciones preliminares: entregar el incremento de software al cliente de modo que la funcionalidad que se haya implementado pueda demostrarse al cliente y éste pueda evaluarla. Es importante tener en cuenta que las demostraciones de vista previa no tienen todas las funcionalidades como estaba previsto, pero se entregará a tiempo. Beedle y sus colegas [Bee99] brindan un análisis exhaustivo de estos modelos y dicen: “Scrum supone de entrada la existencia de caos...” Los modelos de proceso de Scrum permiten a un equipo de software trabaje con éxito en un mundo donde la incertidumbre no se puede eliminar (Pressman, 2010).

2.3. PROCESO UNIFICADO ÁGIL (PUA)

El proceso ágil unificado (AUP) adopta la filosofía de “en serie para lo grande” e “iterativa para lo pequeño” [Amb06] para construir sistemas computarizados. Al adoptar los procesos incrementales clásicos de PU - concepción, elaboración, construcción y transición- PUA ofrece un recubrimiento secuencial (por ejemplo, la secuencia lineal de actividades de ingeniería de software) que permite al equipo visualizar el proceso general del proyecto de software. Sin embargo, en cada actividad, el equipo entrena para ganar agilidad y proporcionar programas útiles a los usuarios finales lo más rápido posible(Pressman, 2010).

Cada iteración de PUA resuelve las siguientes actividades [Amb06]:

- Modelado. Se generan representaciones UML de los dominios del negocio y el problema. Sin embargo, para mantener la flexibilidad, estos modelos deben ser “sólo suficientemente buenos” [Amb06] para permitir que el equipo avance.
- Implementación. Los modelos se compilan en el código fuente.
- Prueba. Al igual que con XP, el equipo diseña y ejecuta una serie de pruebas para detectar errores y asegúrese de que el código fuente cumpla con sus requisitos.
- Despliegue. Como en el proceso general del proceso y la publicación en este contexto se centra en la entrega de software incremental para obtener comentarios de los usuarios finales (Pressman, 2010). (Pressman, 2010)

- Configuración y administración del proyecto. Según la AUP, la gestión de la configuración, incluye la administración del cambio y el riesgo, y el control de cualesquiera productos del trabajo permanente producido por el equipo. La administración del proyecto rastrea y monitorea el progreso del equipo y coordinar sus actividades (Pressman, 2010).

- Administración del ambiente. La Administración del ambiente coordina una infraestructura del proceso, incluidos los estándares, las herramientas y otras técnicas de apoyo disponible para el grupo (Pressman, 2010).

Aunque PUA tiene vínculos históricos y técnicos con lenguaje unificado de modelado, es importante tener en cuenta que el modelado UML se puede utilizar con cualquier modelo para el proceso ágil (Pressman, 2010).

2.4. OTROS

La historia de la ingeniería de software está repleta de docenas de descripciones de procesos, metodologías, métodos de modelado y notación, herramientas y técnicas, todos anticuados. Todos eran famosos, luego eclipsados por algo nuevo y (supuestamente). Con la aparición de muchos modelos de procesos ágiles, cada uno de ellos han estado luchando por ser aceptado en la comunidad de desarrollo de software: el movimiento ágil es el mismo camino histórico (Pressman, 2010).

Los modelos de procesos ágiles más utilizados son Programación Extrema (XP). Pero muchos otros han sido sugeridos y usados todo el tiempo. Entre ellos se encuentran los siguientes:

- Desarrollo adaptativo de software (DAS)
- Scrum
- Método de desarrollo de sistemas dinámicos (MDSD)
- Cristal
- Desarrollo impulsado por las características (DIC)
- Desarrollo esbelto de software (DES)
- Modelado ágil (MA)
- Proceso unificado ágil (PUA)

Es importante notar que todos los modelos de proceso ágil se apegan (en mayor o menor grado) al Manifiesto para el desarrollo ágil de software (Pressman, 2010).

2.4.1. Desarrollo adaptativo de software (DAS)

El desarrollo de software adaptativo (DAS) fue propuesto por Jim Highsmith [Hig00] como una técnica para elaborar software y sistemas complejos. La base filosófica de DAS es que prefieren la cooperación humana y la correcta organización del grupo. Highsmith argumenta que un enfoque colaborativo para el desarrollo adaptativo es "tanto una fuente de orden en nuestras complejas interacciones, como de disciplina e ingeniería". Define el 'ciclo de vida' del DAS (ver ilustración 8), que consta de tres fases: especulación, colaboración y aprendizaje (Pressman, 2010).

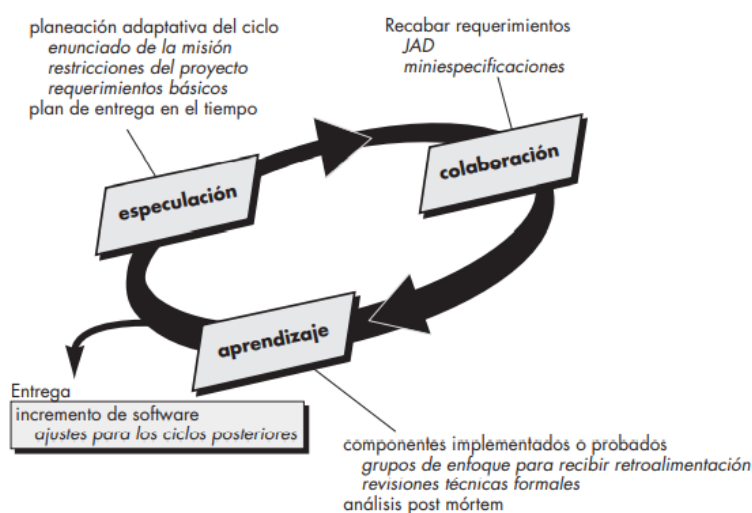


Ilustración 8: Desarrollo adaptativo de software

Fuente. (Pressman, 2010).

Durante la especulación, se lanza el proyecto y se lleva a cabo la planeación adaptativa del proyecto. LA especulación utiliza la información de lanzamiento del proyecto - enunciado de misión de los clientes, restricciones del proyecto (por ejemplo, fechas de entrega o descripciones de usuario) y requerimientos básicos— para definir el conjunto de ciclos de entrega (incrementos de software) serán necesarios para el proyecto. No importa cuán completo y visionario sea el plan del curso, está destinado a cambiar. Con base en la información obtenida al final del primer ciclo, el plan es revisado y ajustado para que el trabajo previsto se adapte mejor a la realidad en la que se desenvuelve el equipo DAS (Pressman, 2010).

Las personas entusiasmadas usan la colaboración de una manera que multiplica su talento y creatividad más allá de sus números absolutos. Este enfoque es un tema recurrente a lo largo de todos los métodos ágiles. Sin embargo, la cooperación no es fácil comprender, incluye comunicación y el trabajo como grupo, pero también promueve la individualidad porque la creatividad individual juega un papel importante en el pensamiento colaborativo. Es sobre todo una cuestión de confianza (Pressman, 2010).

Las personas que trabajan juntas necesitan confiar las unas en las otras para:

- 1) criticarse sin enojo,
- 2) ayudarse sin resentimiento,
- 3) trabajar más o más de lo normal,
- 4) tener las habilidades adecuadas para contribuir al trabajo, y
- 5) comunicar cualquier problema o inquietud manera que conduzcan a la acción efectiva.

Cuando los miembros del equipo de DAS comenzaron a desarrollar componentes como parte del ciclo de adaptación, el enfoque está en “aprendizaje” sobre lo que esté en el avance hacia la terminación del ciclo. De hecho, Highsmith [Hig00] argumenta que los desarrolladores de software a menudo sobreentienden (de tecnología de operaciones y proyectos) y este aprendizaje les ayudará a aumentar su nivel de comprensión realmente. Los equipos de DAS aprenden de tres maneras: grupos de enfoque, evaluaciones técnicas y análisis de proyectos post-mortem (Pressman, 2010).

La filosofía DAS es válida independientemente del modelo de proceso utilizado. Confirma lo que hace DAS depende de la dinámica del grupo con su propia organización y colaboración en el aprendizaje personal, individual y grupal crea equipos para proyectos de software que tienen mayores posibilidades de éxito (Pressman, 2010).

2.4.2. Método de desarrollo de sistemas dinámicos (MDSD)

El método de desarrollo de sistemas dinámicos (MDSD) [Sta97] ⁴es un enfoque de desarrollo ágil de software que proporciona un marco para construir y mantener sistemas que cumpla con las limitaciones de tiempo a través de prototipos adicionales en un entorno de proyecto controlado” [CCS02] ⁵(Pressman, 2010).

La filosofía MDSD se toma de una versión revisada de la Regla de Pareto: el 80% de las solicitudes se pueden entregar en 20% del porcentaje de tiempo que lleva entregarlo completo (100 por ciento) (Pressman, 2010).

MDSD es un proceso de programación iterativo en el que cada iteración sigue la regla del 80 %. se requiere sólo suficiente trabajo para cada incremento con objeto de facilitar el paso al siguiente. Los detalles restantes se finalizarán más adelante, cuando se conozcan las necesidades comerciales y se hayan solicitado e implementado los cambios (Pressman, 2010).

DSDM Corporation (www.dsdm.org) es un grupo global de empresas que asume el papel general de "custodios" del método. El consorcio ha definido el modelo de proceso ágil, conocido como ciclo de vida MDSD, en la que identifica tres ciclos iterativos distintos, antes de dos procesos adicionales del ciclo de vida: (Pressman, 2010).

Estudio de factibilidad: Establecimiento de requisitos básicos y restricciones relacionadas con la aplicación que se construirá, para evaluar si la aplicación es un candidato viable para el proceso MDSD (Pressman, 2010).

Estudio del negocio: determinación de requisitos funcionales y voluntad de información que permite a las aplicaciones agregar valor al negocio; Asimismo, define la infraestructura de la aplicación y define los requisitos para su mantenimiento (Pressman, 2010).

Iteración del modelo funcional: Produce un conjunto incremental de prototipos para demostrar la funcionalidad a los clientes. (Nota: Todos los modelos MDSD son para que evolucionen hacia la aplicación que se entrega). El objetivo de este ciclo iterativo es cumplir con los requisitos adicionales al obtener comentarios de los usuarios mientras practican el prototipo (Pressman, 2010).

⁴ Stapleton, J., DSDM—Dynamic System Development Method: The Method in Practice, Addison-Wesley, 1997.

⁵ CS3 Consulting Services, 2002, www.cs3inc.com/DSDM.htm.

Diseño e iteración de la construcción: hace una revisión de prototipos integrados en la iteración del modelo funcional para asegurar que cada iteración se diseñe correctamente a en forma que permita dar valor operativo del negocio a los usuarios finales; la iteración del modelo funcional y el diseño e iteración de la construcción ocurren simultáneamente (Pressman, 2010).

Implementación: coloque la última versión del software (un prototipo "funcional") en el entorno real. Cabe señalar que:

- 1) No está permitido ganar 100%
- 2) Es posible que se requieran cambios al realizar una actualización.

En ambos casos, el trabajo de desarrollo de MDSD continua y vuelve a las iteraciones de modelado funcional (Pressman, 2010).

MDSD se combina con XP para proporcionar un enfoque común y definir un modelo sólido del proceso (ciclo de vida MDSD) con prácticas detalladas (XP) necesarios para generar incrementos en los programas. Además, los conceptos de DAS se ajustan a un modelo combinado del proceso (Pressman, 2010).

2.4.3. Cristal

Alistar Cockburn [Coc05] creó la familia Cristal de métodos ágiles, para conseguir eEl desarrollo de software premia el ""maniobrabilidad"" en lo que describe Cockburn caracteriza como "un juego cooperativo con recursos, inventos y conexiones limitados, cuyo objetivo principal es proporcionar un software útil que funcione y el objetivo secundario es sugerir el próximo juego" [Coc02] (Pressman, 2010).

Para lograr la maniobrabilidad, Cockburn y Highsmith identificaron un conjunto de enfoques, cada uno con elementos básicos comunes, roles y modelos de procesos, productos y prácticas individuales de trabajo. La verdadera familia Crystal es un conjunto de ejemplos de procesos ágiles que se ha demostrado que funcionan para diferentes tipos de proyectos.

El objetivo es permitir que los equipos ágiles seleccionen al miembro más apropiado de la familia Crystal para su proyecto y entorno (Pressman, 2010).



Recuerde que. Cristal es una familia de modelos de proceso con el mismo “código genético” pero diferentes métodos para adaptarse a las características del proyecto.

2.4.4. Desarrollo impulsado por las características (DIC)

El desarrollo impulsado por características (DIC) fue diseñado originalmente por Peter Coad y Fellows [Coa99] como un modelo de proceso práctico para la ingeniería orientada a objeto. Stephen Palmer y John Felsing [Pal02] ⁶ampliaron y mejoraron el trabajo de Coad describiéndole a un proceso adaptativo y ágil aplicable a proyectos de software medianos y grandes. Al igual que otros proyectos ágiles, DIC adopta la filosofía de: 1) Enfatizar la colaboración entre los miembros del equipo DIC. 2) gestionar la complejidad de los problemas y del proyecto mediante análisis basado en características, seguido de integración de software incremental, y 3) comunicar verbal y gráficamente los detalles técnicos con medios basados de texto. DIC se centra en las actividades de aseguramiento para la calidad del software fomentando estrategias de desarrollo incremental, utilizando pruebas de diseño y código, aplicación de prueba de control de calidad de software, conjunto de métricas y uso de modelos (para el análisis, diseño y construcción).

En el contexto de DIC, la característica “es una función preferida por el cliente que puede

Tomado en dos semanas o menos” [Coa99]. Se centrar en la definición de características que proporciona las siguientes ventajas:

- Dado que las características son pequeños bloques funcionales proporcionados, de manera fácil para que los usuarios describan y entiendan cómo se relacionan entre sí y las revisan mejor en busca de ambigüedades, errores u omisiones.
- Las características se organizan por jerarquía o grupos vinculados a la empresa.
- Como característica se proporciona un aumento en el programa DIC, el dispositivo, el equipo desarrollar características de rendimiento cada dos semanas.
- El diseño y representación en código de las características son más fáciles de inspeccionar con eficacia porque éstas son pequeñas.

⁶ Palmer, S. y J. Felsing, A Practical Guide to Feature Driven Development, Prentice Hall, 2002.

- La planificación y programación de actividades y el seguimiento está determinada por jerarquía de puestos, y no a través de un conjunto de tareas de ingeniería de software adoptado arbitrariamente.

Coad y colegas [Coa99] recomiendan el siguiente diagrama para la determinación del trabajo:

<acción> el <resultado> <a|por|de|para> un <objeto>

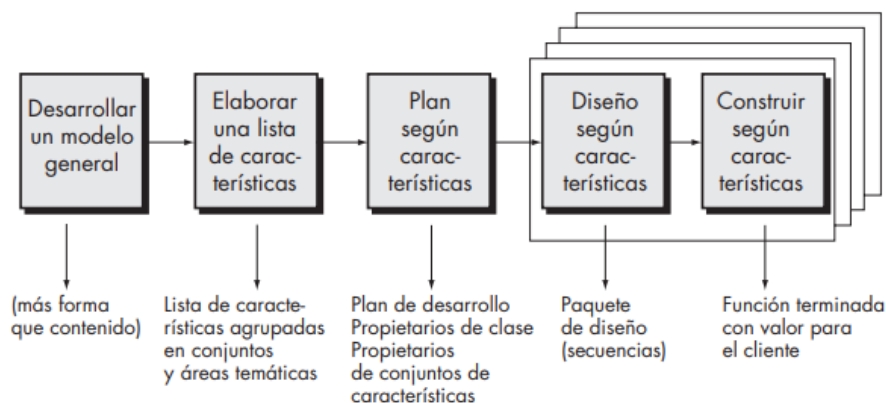


Ilustración 9: Desarrollo impulsado por las características [Coa99] (con permiso)

Fuente: (Pressman, 2010).

Donde <objeto> es "una persona, lugar o cosa (incluido un rol, tiempo o período de tiempo o descripciones temporales o similares a entradas de catálogo)". Algunos ejemplos de las funciones de la aplicación de comercio electrónico son las siguientes: (Pressman, 2010).

Agregar el producto al carrito de compras

Mostrar las especificaciones técnicas del producto

Guardar la información de envío para el cliente

La agrupación de características congrega características similares en categorías relacionadas a una empresa y se define como: (Pressman, 2010).

<acción><ndo> un <objeto>

Ejemplo: Vender un producto es un conjunto de propiedades a recopilar. El enfoque de DIC identifica cinco actividades estructurales "colaborativas" [Coa99], DIC se denomina "proceso"), como se muestra en la ilustración 9 (Pressman, 2010).

DIC se centra más que otros métodos ágiles en las directrices y técnicas de gestión de proyectos. Con el tamaño y la complejidad cada vez mayores de estas cosas, no es de extrañar que la gestión de proyectos, en particular, tiene muchas deficiencias. Para los desarrolladores, sus gerentes y otras partes interesadas, es esencial comprender el estado del proyecto, es decir, se ha avanzado y surgen problemas. Si la presión para cumplir con el plazo es grande, es importante determinar si la entrega del complemento está planificada correctamente. Para lograrlo, DIC identifica seis puntos de referencia en el diseño e implementación de una característica: “recorrido por el diseño, diseño y prueba paso a paso de diseño, código, inspección del código, decisión de construir” [Coa99] ⁷ (Pressman, 2010).

2.4.5. Desarrollo esbelto de software (DES)

El desarrollo esbelto de software (DES) trae los principios de manufactura al mundo de ingeniería de software. Los principios de esbeltez que inspiran al proceso DES se resumen como sigue ([Pop03], [Pop06a]): eliminar el desperdicio, generar calidad, crear conocimiento, aplazar el compromiso, entregar rápido, respetar a las personas y optimizar al todo. Cada uno de estos principios se puede personalizar para el proceso de software. Por ejemplo, eliminar el desperdicio en un proyecto ágil [Das05] ⁸significa: 1) No más características o funciones redundantes, 2) una evaluación del costo y la eficacia que tendrán los nuevos requisitos necesarios en el cronograma operativo, 3) la eliminación de todos los pasos, 4) crear mecanismos para mejorar la forma en que los miembros del equipo obtienen información, 5) asegurar que las capturas se prueben con tantos errores como sea posible, 6) reducir el tiempo requerido para solicitar y obtener una decisión que afecte el programa o el proceso utilizado para crearlo, y 7) simplificar la forma en que se comunica la información a todos los participantes en el proceso (Pressman, 2010).

2.4.6. Modelado ágil (MA)

Hay muchas situaciones en las que los ingenieros de software necesitan construir grandes sistemas de gran importancia para la empresa. El alcance y la complejidad de estos sistemas

⁷ Coad, P., E. Lefebvre y J. DeLuca, Java Modeling in Color with UML, Prentice Hall, 1999.

⁸Dasari, R., “Lean Software Development”, a white paper, descargable de www.projectperfect.com.au/downloads/Info/info_lean_development.pdf, 2005.

deben modelarse para que: 1) todos los actores comprendan mejor lo que se debe hacer y queden satisfecho, 2) el problema puede ser efectivamente distribuido a quienes tienen que resolverlo, y 3) Calidad asegurada en el diseño y construcción del sistema (Pressman, 2010).

En los últimos 30 años, se han propuesto muchos métodos de notación y modelado para que la arquitectura de software realice análisis y diseño (tanto en arquitectura como en componentes). Estos métodos tienen sus ventajas, pero son difícil de adoptar y mantener (en muchos proyectos). Parte del problema es el "peso" de estos métodos modelados. Con esto se hace referencia al volumen de la notación que se requiere, al grado de formalismo sugerido, el tamaño absoluto de los modelos de proyecto grandes y difícil de mantener la(s) forma(s) cuando ocurren cambios. Sin embargo, el análisis de diseño y el modelado tienen muchas ventajas para proyectos grandes, solo porque hace que estos proyectos sean más manejables intelectualmente (Pressman, 2010).

¿Existe un enfoque ágil para el modelado de ingeniería de software que proporciona una alternativa? En el "sitio oficial de modelado ágil", Scott Ambler [Amb02a] describe el modelo ágil (AM) del modo siguiente: (Pressman, 2010).

El modelado ágil (MA) es una metodología basada en la práctica para modelar y documentar con eficacia los sistemas basados en software. En pocas palabras, es un conjunto de valores, principios y prácticas para hacer modelos de software aplicables de manera eficaz y ligera a un proyecto de desarrollo de software. Los modelos ágiles son más eficaces que los tradicionales porque son sólo buenos, sin pretender ser perfectos(Pressman, 2010).

El modelo Agile abarca todos los valores del manifiesto ágil. filosofía de modelado ágil establece que un equipo ágil debe tener el coraje de tomar decisiones sobre el rechazo de un diseño y reconstruirlo. El equipo también debe reconocer humildemente que los tecnólogos no tienen todas las respuestas y que los profesionales de negocios y otras partes interesadas deben ser respetado e incluido (Pressman, 2010).

Aunque MA recomienda varios principios de modelado "fundamentales" y "suplementarios", aquellos que son exclusivos del MA son los siguientes [Amb02a]: (Pressman, 2010).

Modelo con un propósito. Un desarrollador que usa MA debe tener en cuenta el propósito específico (p. ej., comunicar información a los clientes o ayudarlos a comprender mejor ciertos aspectos de un programa) antes del modelado. Una vez que estableces la meta

para el modelo, el tipo y la resolución de los símbolos utilizados serán más claros (Pressman, 2010).

Uso de modelos múltiples. Hay muchos modelos y símbolos diferentes posibles de utilizar para describir programas. Para la mayoría de los proyectos, sólo uno subgrupo es esencial un pequeño subconjunto. MA sugiere que, para proporcionar la perspectiva necesaria, cada modelo presenta un aspecto diferente del sistema y solo se deben usar esos modelos para agregar valor al público objetivo (Pressman, 2010).

Viajar ligero. A medida que avanza el trabajo de ingeniería de software, conserve solo los modelos con valor a largo plazo y deseche el resto. Cualquier producto de trabajo que se mantenga debe mantenerse cuando se realicen cambios. Esto representa que ralentiza el trabajo del grupo. Ambler [Amb02a] ⁹ dice “cada vez que se decide conservar un modelo, se pierde agilidad en nombre de la conveniencia de tener disponible esa información en forma abstracta para el equipo (y de ese modo mejorar potencialmente la comunicación dentro del equipo, así como con los participantes)” (Pressman, 2010).

El contenido es más importante que la representación. El modelado debe continuar con la Información para el público objetivo. Excelente ejemplo de una pequeña sintaxis para el envío con contenido útil no es tan valioso como otro contenido categorizado incorrectamente, pero, sin embargo, proporcione contenido valioso a los usuarios (Pressman, 2010).

Conocer los modelos y herramientas que se utilizan en su creación. Comprender los puntos fuertes y débiles de cada modelo y las herramientas utilizadas para crearlos (Pressman, 2010).

Adaptación local. El enfoque de modelado debe adaptarse a las necesidades del equipo ágil (Pressman, 2010).

Una parte importante de la comunidad de ingeniería de software utiliza el lenguaje Modelado unificado (UML) como una forma preferida de representar modelos de análisis y diseño. El Proceso Unificado fue desarrollado para proporcionar la estructura para la aplicación UML (Pressman, 2010).

⁹ Ambler, S., “What Is Agile Modeling (AM)?” 2002, www.agilemodeling.com/index.htm.

3. HERRAMIENTAS PARA LA GESTIÓN ÁGIL.

Algunos defensores de la filosofía ágil argumentan que las herramientas de software automatizadas (por ejemplo, diseños) deben considerarse como una adición secundaria a las actividades del equipo y no la base del éxito. Sin embargo, Alistair Cockburn [Coc04] ¹⁰ sugiere que las herramientas tienen un beneficio y que “los equipos ágiles priorizan el uso de las herramientas que permiten una rápida comprensión. Algunas de estas herramientas son sociales e inician a partir de la etapa de reclutamiento. Otros son tecnología y ayudan a que los equipos distribuidos imitan su presencia real. Muchas herramientas son físicas y permiten que la gente lo manipule en los talleres. Casi todos los modelos de procesos ágiles son un factor clave en la contratación de empleo (reclutamiento), colaboración en equipo, comunicación con los participantes y gestión indirecta; Por ello, Cockburn afirma que las "herramientas" que abordan estos aspectos son un factor clave en el éxito de la agilidad; por ejemplo, una “herramienta” de reclutamiento tal vez sea el requerimiento de que un prospecto a miembro del equipo pase algunas horas programando en pareja con alguien que ya es integrante del equipo (Pressman, 2010).

La conformidad se evalúa inmediatamente. Las "herramientas" de colaboración y comunicación suelen ser de baja tecnología y aplicar una combinación de cualquier mecanismo (“distancia física, pizarra, pizarra, tarjetas y notas adhesivas” [Coc04] que proporciona información y sinergias entre desarrolladores ágiles (Pressman, 2010).

La comunicación activa se logra a través de dinámicas de grupo (como la programación en parejas), mientras que la comunicación pasiva se logra a través de “radiadores de información” (un tablero que muestre el estado general de los distintos componentes de un incremento). Las herramientas de administración de proyectos no confirman el gráfico de diagrama de Gantt y su reemplazo con otros elementos de valor agregado o 'prueba generada contra un diagrama aprobado'; otras herramientas ágiles utilizadas para mejorar el entorno en el que trabaja el equipo ágil (por ejemplo, lugares de reunión más eficientes), mejorando la cultura del equipo a través del desarrollo de interacciones sociales (grupos que tienen algo en común), equipamiento físico (mesas electrónica) y mejora de procesos (como programación de pares o facturación de tiempo)” [Coc04] ¿Alguna de las ya mencionadas

¹⁰ Cockburn, A., “What the Agile Toolbox Contains”, CrossTalk, noviembre 2004, disponible en www.stsc.hill.af.mil/crosstalk/2004/11/0411Cockburn.html

son herramientas? Así es, si facilitan el trabajo implementado por un miembro del equipo ágil y mejora la calidad del producto final (Pressman, 2010).

3.1. JIRA - SOFTWARE GESTIÓN DE PROYECTOS

JIRA es un producto creado por Atlassian y lanzado en octubre del año 2004. Se trata de una aplicación web que la empresa comercializa mediante la venta de derechos de uso. El precio de los derechos depende del número de usuarios que los utilizarán la aplicación. Atlassian Web App es un gestor de proyectos que permite a los equipos de proyectos planificar, construir y finalizar proyectos. Además, te permite capturar y organizar incidencias, asignar trabajo y hacer un seguimiento al desempeño del equipo (Trabajo & Grado, 2015)

Las principales características que ofrece la aplicación son:

- Gestión de tareas: el usuario puede crear una tarea con toda la información requeridas, priorizando las tareas existentes, asignándolas a los usuarios y mantenerse al día de la evolución de las mismas (Trabajo & Grado, 2015)- Crear flujo de trabajo: proporciona flujo de trabajo y también le permite crear nuevo flujo de trabajo de la empresa, que compra Jira, puede dar seguimiento por la aplicación (Trabajo & Grado, 2015)

Planificación de proyectos: permite planificar todo el trabajo a realizar dentro del proyecto, asignando y priorizando cada tarea. También puede adaptarse a metodologías que evolucionan rápidamente (Trabajo & Grado, 2015)

- Colaboración entre grupos: Facilitar la colaboración entre los miembros del equipo y colaboración entre equipos (Trabajo & Grado, 2015)

- Tener visibilidad y notificación de todo lo que le importa al usuario: esto permite configurar qué notificaciones quiere recibir el usuario y por qué canal se encuentra y quiere recibir (email, chat o móvil). De esta forma, el usuario siempre será notificado (Trabajo & Grado, 2015)

- Gestión de proyectos: Desde la aplicación podrás gestionar todo lo relacionado con el mismo proyecto sin tener que usar nada más. Además, existe la posibilidad de gestionar proyectos que quieren utilizar métodos flexibles (Trabajo & Grado, 2015)

- Interfaz móvil: proporciona una interfaz móvil para que el equipo esté siempre actualizado en cualquier momento, en cualquier lugar. Funciona a través del navegador. Entonces no necesitas instalar nada en el teléfono móvil (Trabajo & Grado, 2015)

- Gestión integrada de repositorios de códigos: desde la app, los desarrolladores del equipo pueden cambiar entre tareas, código y solicitud. Esta transición se centra en las tareas relacionadas con el escenario. Entonces el programador puede enfocarse en el desarrollo y nada más. Además, la herramienta gestiona repositorio de código mediante la creación automáticamente de las acciones necesarias para mantener el código del proyecto en buen estado, bien recopilado y organizado (Trabajo & Grado, 2015)

- Capacidad para importar datos de múltiples herramientas de planificación de proyectos: Esta función es útil si se utilizó otro administrador de tareas antes de que se publicara JIRA. De esta forma, la empresa no pierde datos anteriores y puede continuar donde lo dejaste (Trabajo & Grado, 2015)

Brindan servicio de importación de 8 herramientas (Bugzilla, FogBugz, Trac, Redmine, Mantis, IBM, HP, GitHub) y también hay una opción para "otros". entre ellos, por cierto, si no puede encontrar sus herramientas antiguas, le ayudará a ingresar los datos en su caso particular (Trabajo & Grado, 2015).

BIBLIOGRAFÍA

- M. Gómez, J. Cervantes, & P. González. (2019). Fundamentos de Ingeniería de Software. In *Jurnal Ekonomi Malaysia* (Vol. 51, Issue 2).
- Patricia, J., Gamboa, Z., Alexandra, C., & Arreaga, L. (2018). Evolución de las Metodologías y Modelos utilizados en el Desarrollo de Software. Evolution of the Methodologies and Models used in Software Development. *INNOVA Research Journal*, 3(10), 20–33.
- Pressman, R. S. (2010). *Ingeniería del Software Un enfoque práctico* (Séptima Ed). McGraw-Hill. <http://cotana.informatica.edu.bo/downloads/Id-Ingenieria.de.software.enfoque.practico.7ed.Pressman.PDF>
- Sommerville, J. A. N. (2005). *Ingeniería del software Ingeniería del software Séptima edición*.
- Trabajo, M., & Grado, F. De. (2015). *Implantación JIRA*.